Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-03-26

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting and dimensional analysis (e.g., **Pint**)

AGH

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting and dimensional analysis (e.g., **Pint**)

- **reproducibility** in computational science (and beyond)
- automation & reuse (DRY)
- **SciPy** (but what about SciPy+Pint?!)

AGH

previously on...

- what (or who) is **RSE**?
- Python as a glue language
- unit-aware coding/plotting and dimensional analysis (e.g., **Pint**)

- **reproducibility** in computational science (and beyond)
- automation & reuse (DRY)
- **SciPy** (but what about SciPy+Pint?!)

- Python's "duck typing" (e.g., glueing SciPy with Pint)
- JIT-compilation of Python code (e.g., using Numba)

AGH

plan for today

- formal requirements for scientific software: PLoS ONE example
- chosing a license (code vs. content; copyleft vs. permissive)
- versioning (semantic), releasing (GitHub) and persistent archival (Zenodo)
- packaging Python code (pyproject.toml) – code along!

AGH

plan for today

- **formal requirements for scientific software: PLoS ONE example**
- chosing a license (code vs. content; copyleft vs. permissive)
- versioning (semantic), releasing (GitHub) and persistent archival (Zenodo)
- packaging Python code (pyproject.toml) – code along!

PLOS One

PLOS One (stylized ***PLOS ONE***, and formerly ***PLoS ONE***) is a peer-reviewed open access mega journal published by the Public Library of Science (PLOS) since 2006. The journal covers primary research from any discipline within science and medicine. The Public Library of Science began in 2000 with an online petition initiative by Nobel Prize winner Harold Varmus, formerly director of the National Institutes of Health and at that time director of Memorial Sloan–Kettering Cancer Center; Patrick O. Brown, a biochemist at Stanford University; and Michael Eisen, a computational biologist at the University of California, Berkeley, and the Lawrence Berkeley National Laboratory.

Submissions are subject to an article processing charge and, according to the journal, papers are not to be excluded on the basis of lack of perceived importance or adherence to a scientific field. All submissions go through a pre-publication review by a member of the board of academic editors, who can elect to seek an opinion from an external reviewer. In January 2010, the journal was included in the *Journal Citation Reports* and received its first impact factor of 4.4. Its 2024 impact factor is 2.6. *PLOS One* papers are published under Creative Commons licenses.

PLOS ONE

PLOS ONE
A peer-reviewed, open access journal

Discipline	Multidisciplinary
Language	English
Edited by	Emily Chenette
Publication details	
History	2006
Publisher	Public Library of Science
Frequency	Upon acceptance
Open access	Yes
License	Creative Commons Attribution License 4.0 International
Impact factor	2.6 (2024)

AGH

Methods, software, databases, and tools

PLOS One will consider submissions that present new methods, software, databases, or tools as the primary focus of the manuscript if they meet the following criteria:

Utility
The tool must be of use to the community and must present a proven advantage over existing alternatives, where applicable. Recapitulation of existing methods, software, or databases is not useful and will not be considered for publication. Combining data and/or functionalities from other sources may be acceptable, but simpler instances (i.e. presenting a subset of an already existing database) may not be considered. For software, databases, and online tools, the long-term utility should also be discussed, as relevant. This discussion may include maintenance, the potential for future growth, and the stability of the hosting, as applicable.

Validation
Submissions presenting methods, software, databases, or tools must demonstrate that the new tool achieves its intended purpose. If similar options already exist, the submitted manuscript must demonstrate that the new tool is an improvement over existing options in some way. This requirement may be met by including a proof-of-principle experiment or analysis; if this is not possible, a discussion of the possible applications and some preliminary analysis may be sufficient.

Availability
If the manuscript's primary purpose is the description of new software or a new software package, this software must be open source, deposited in an appropriate archive, and conform to the Open Source Definition. If the manuscript mainly describes a database, this database must be open-access and hosted somewhere publicly accessible, and any software used to generate a database should also be open source. If relevant, databases should be open for appropriate deposition of additional data. Dependency on commercial software such as Mathematica and MATLAB does not preclude a paper from consideration, although complete open source solutions are preferred. In these cases, authors should provide a direct link to the deposited software or the database hosting site from within the paper. If the primary focus of a manuscript is the presentation of a new tool, such as a newly developed or modified questionnaire or scale, it should be openly available under a license no more restrictive than CC BY.

Software submissions

Manuscripts whose primary purpose is the description of new software must provide full details of the algorithms designed. Describe any dependencies on commercial products or operating system. Include details of the supplied test data and explain how to install and run the software. A brief description of enhancements made in the major releases of the software may also be given. Authors should provide a direct link to the deposited software from within the paper.

plan for today

- formal requirements for scientific software: PLoS ONE example
- **chosing a license (code vs. content; copyleft vs. permissive)**
- versioning (semantic), releasing (GitHub) and persistent archival (Zenodo)
- packaging Python code (pyproject.toml) – code along!

About CC Licenses

Creative Commons licenses give everyone from individual creators to large institutions a standardized way to grant the public permission to use their creative work under copyright law. From the reuser's perspective, the presence of a Creative Commons license on a copyrighted work answers the question, *What can I do with this work?*

Licenses and Tools

About CC Licenses

Use & remix

FAQ

Technology Platforms

Made with Creative Commons

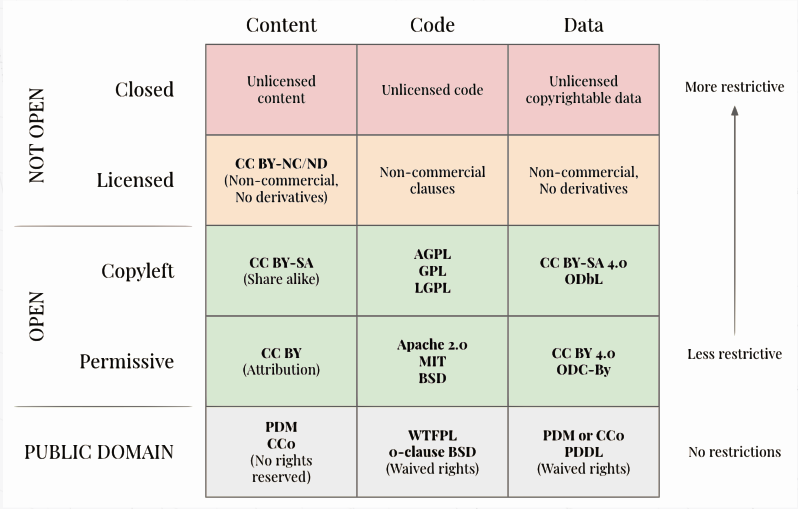
Creative Commons for code?

Can I apply a Creative Commons license to software?

We recommend against using Creative Commons licenses for software. Instead, we strongly encourage you to use one of the very good software licenses which are already available. We recommend considering licenses listed as free by the Free Software Foundation and listed as "open source" by the Open Source Initiative.

Unlike software-specific licenses, CC licenses do not contain specific terms about the distribution of source code, which is often important to ensuring the free reuse and modifiability of software. Many software licenses also address patent rights, which are important to software but may not be applicable to other copyrightable works. Additionally, our licenses are currently not compatible with the major software licenses, so it would be difficult to integrate CC-licensed work with other free software. Existing software licenses were designed specifically for use with software and offer a similar set of rights to the Creative Commons licenses.

choosing a license



		Content	Code	Data	
NOT OPEN	Closed	Unlicensed content	Unlicensed code	Unlicensed copyrightable data	More restrictive
	Licensed	**CC BY-NC/ND** (Non-commercial, No derivatives)	Non-commercial clauses	Non-commercial, No derivatives	
OPEN	Copyleft	**CC BY-SA** (Share alike)	**AGPL GPL LGPL**	**CC BY-SA 4.0 ODbL**	
	Permissive	**CC BY** (Attribution)	**Apache 2.0 MIT BSD**	**CC BY 4.0 ODC-By**	Less restrictive
PUBLIC DOMAIN		**PDM CCo** (No rights reserved)	**WTFPL 0-clause BSD** (Waived rights)	**PDM or CCo PDDL** (Waived rights)	No restrictions

https://agilescientific.com/blog/2021/2/17/which-open-licence-should-i-choose

ETHICS, LAW, COMPUTER SOFTWARE

Free Software, Free Society

Selected Essays of Richard M. Stallman
Second Edition

This book collects the writing of Richard Stallman in a manner that will make its subtlety and power clear. The essays span a wide range, from copyright to the history of the free software movement. They include many arguments not well known, and among these, an especially insightful account of the changed circumstances that render copyright in the digital world suspect. They will serve as a resource for those who seek to understand the thought of this most powerful man—powerful in his ideas, his passion, and his integrity, even if powerless in every other way. They will inspire others who would take these ideas, and build upon them.
from the foreword, by Lawrence Lessig

Richard M. Stallman is an internationally recognized computer programmer, political activist, and author. In 1983, he founded the free software movement—an organized effort to protect computer users' freedom—by launching the GNU Project, which seeks to put together a body of software sufficient to end users' dependence on proprietary software.

He coined the term "copyleft" and is the main author of several copyleft licenses, including the most widely used free software license, the GNU General Public License, which guarantees the four freedoms to all users of software placed under it: the freedoms to run, to study, to modify, and to redistribute the program. The GNU/Linux System (basically the GNU operating system with the kernel Linux added) is today used on tens of millions of computers.

Stallman is the recipient of numerous awards, including the ACM Grace Hopper Award, a MacArthur Foundation fellowship, the Electronic Frontier Foundation's Pioneer Award, and the Takeda Award for Social and Economic Well-Being.

Stallman's biography, *Free as in Freedom,* by Sam Williams, is also published by GNU Press.

FREE SOFTWARE FOUNDATION

GNU Press

Proceeds from the sale of this book fund our efforts to preserve, protect, and promote software freedom.

Free Software, Free Society
Selected Essays of Richard M. Stallman, Second Edition

FREE SOFTWARE FOUNDATION

GNU Press

Free Software, Free Society

Selected Essays of Richard M. Stallman
Second Edition

By Richard M. Stallman
Foreword by Lawrence Lessig

AGH

RMS on *Releasing Free Software If You Work at a University*

https://www.gnu.org/philosophy/university.html

RMS on *Releasing Free Software If You Work at a University*

https://www.gnu.org/philosophy/university.html

- *...principle: does the university have a mission to* **advance human knowledge**, *or is its sole purpose to perpetuate itself?*

AGH

RMS on *Releasing Free Software If You Work at a University*

https://www.gnu.org/philosophy/university.html

- *...principle: does the university have a mission to* **advance human knowledge**, *or is its sole purpose to perpetuate itself?*
- *...***treat the public ethically**, *the software should be free — as in freedom — for the whole public.*

AGH

RMS on *Releasing Free Software If You Work at a University*

https://www.gnu.org/philosophy/university.html

- *...principle: does the university have a mission to* **advance human knowledge**, *or is its sole purpose to perpetuate itself?*
- *...***treat the public ethically**, *the software should be free — as in freedom — for the whole public.*
- *...allowing others to share and change software as an expedient for making software* **powerful and reliable**.

AGH

RMS on *Releasing Free Software If You Work at a University*

https://www.gnu.org/philosophy/university.html

- *...principle: does the university have a mission to* **advance human knowledge**, *or is its sole purpose to perpetuate itself?*
- *...***treat the public ethically**, *the software should be free — as in freedom — for the whole public.*
- *...allowing others to share and change software as an expedient for making software* **powerful and reliable**.
- *...GNU General Public License (***GNU GPL***)...*

AGH

https://choosealicense.com/licenses/

GNU GPLv3

Permissions of this strong copyleft license are conditioned on making available complete source code of licensed works and modifications, which include larger works using a licensed work, under the same license. Copyright and license notices must be preserved. Contributors provide an express grant of patent rights.

Permissions

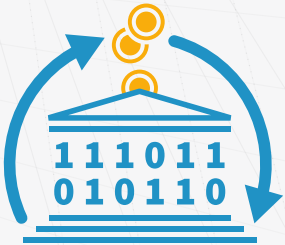
- ✓ Commercial use
- ✓ Distribution
- ✓ Modification
- ✓ Patent use
- ✓ Private use

Conditions

- ⓘ Disclose source
- ⓘ License and copyright notice
- ⓘ Same license
- ⓘ State changes

Limitations

- × Liability
- × Warranty



PUBLIC MONEY

PUBLIC CODE

Why is software created using taxpayers' money not released as Free Software?

We want legislation requiring that publicly financed software developed for the public sector be made publicly available under a Free and Open Source Software (https://fsfe.org/freesoftware/) licence. If it is public money, it should be public code as well.

AGH

plan for today

- formal requirements for scientific software: PLoS ONE example
- chosing a license (code vs. content; copyleft vs. permissive)
- **versioning (semantic), releasing (GitHub) and persistent archival (Zenodo)**
- packaging Python code (pyproject.toml) – code along!

Semantic Versioning 2.0.0

Summary

Given a version number MAJOR.MINOR.PATCH, increment the:

1. MAJOR version when you make incompatible API changes
2. MINOR version when you add functionality in a backward compatible manner
3. PATCH version when you make backward compatible bug fixes

Additional labels for pre-release and build metadata are available as extensions to the MAJOR.MINOR.PATCH format.

Introduction

In the world of software management there exists a dreaded place called “dependency hell.” The bigger your system grows and the more packages you integrate into your software, the more likely you are to find yourself, one day, in this pit of despair.

persistent archival of GitHub (or non-GitHub) releases

Issuing a persistent identifier for your repository with Zenodo

To make your repositories easier to reference in academic literature, you can create persistent identifiers, also known as Digital Object Identifiers (DOIs). You can use the data archiving tool Zenodo to archive a repository on GitHub and issue a DOI for the archive.

> Tip
>
> - Zenodo can only access public repositories, so make sure the repository you want to archive is public.
> - If you want to archive a repository that belongs to an organization, the organization owner may need to approve access for the Zenodo application.
> - Make sure to include a license in your repository so readers know how they can reuse your work.

1. Navigate to the login page for Zenodo.
2. Click **Log in with GitHub**.
3. Review the information about access permissions, then click **Authorize zenodo**.
4. Navigate to the Zenodo GitHub page.
5. To the right of the name of the repository you want to archive, toggle the button to **On**.

Zenodo archives your repository and issues a new DOI each time you create a new GitHub release. Follow the steps at Managing releases in a repository to create a new one.

doi® Foundation

The Identifier

WHAT IS A DOI?

A DOI is a digital identifier of an object, any object — physical, digital, or abstract. DOIs solve a common problem: keeping track of things. Things can be matter, material, content, or activities.

A DOI is a unique number made up of a prefix and a suffix separated by a forward slash. This is an example of one: 10.1000/182. It is resolvable using our proxy server by displaying it as a link: https://doi.org/10.1000/182.

Designed to be used by humans as well as machines, DOIs identify objects persistently. They allow things to be uniquely identified and accessed reliably. You know what you have, where it is, and others can track it too.

AGH

Zenodo.org (CERN)

Zenodo - Research. Shared.

Infrastructure

Organisational

Host institution

Zenodo is hosted by CERN which has existed since 1954 and currently has an experimental programme defined for the next 20+ years. CERN is a memory institution for High Energy Physics and renowned for its pioneering work in Open Access. Organisationally Zenodo is embedded in the IT Department, Collaboration Devices and Applications Group, Digital Repositories Section (IT-CDA-DR).

Zenodo is offered by CERN as part of its mission to make available the results of its work (CERN Convention, Article II, §1).

Legal status

CERN is an intergovernmental organisation and has legal personality in the metropolitan territories of all CERN Member States (CERN Convention, Article IX) and enjoys the corresponding legal capacity under public international law.

plan for today

- formal requirements for scientific software: PLoS ONE example
- chosing a license (code vs. content; copyleft vs. permissive)
- versioning (semantic), releasing (GitHub) and persistent archival (Zenodo)
- **packaging Python code (pyproject.toml) – code along!**

minimal Python package

⇝ **pyproject.toml**

```toml
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "mc_pi"
version = "0.1.0"
license = { text = "GPL-3.0-or-later" }

[tool.setuptools]
py-modules = ["mc_pi"]
```

⇝ **mc_pi.py**

```python
from random import random as u01
pi = lambda n: 4 * sum(abs(u01() + 1j * u01()) < 1 for _ in range(n)) / n
```

installing from Colab

```
!pip install git+https://github.com/slayoo/test.git
```

```
Collecting git+https://github.com/slayoo/test.git
  Cloning https://github.com/slayoo/test.git to /tmp/pip-req-build-5896ozcg
  Running command git clone --filter=blob:none --quiet https://github.com/slayoo/test.git /tmp/pip-req-build-5896ozcg
  Resolved https://github.com/slayoo/test.git to commit 71ceedb787f9866a62f5cff7ac3695cfcd745f18
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: mc_pi
  Building wheel for mc_pi (pyproject.toml) ... done
  Created wheel for mc_pi: filename=mc_pi-0.1.0-py3-none-any.whl size=1159 sha256=87d200ff897eb5d9b99a228b6c63c4357017c3dd5ebde9ef45
  Stored in directory: /tmp/pip-ephem-wheel-cache-3edtwt6w/wheels/5a/e9/02/dbdb138a19a7b090ed3e91fcaf6a9fda79db17be2da257981f
Successfully built mc_pi
Installing collected packages: mc_pi
Successfully installed mc_pi-0.1.0
```

```
from mc_pi import pi
f"{pi(2**22)=:.4g}"
```

```
'pi(2**22)=3.142'
```

```
Start coding or generate with AI.
```

take-home messages: for others to use your code, they need:

take-home messages: for others to use your code, they need:

- to be able to identify the release (versioning)

take-home messages: for others to use your code, they need:

- to be able to identify the release (versioning)
- to locate the code (persistent archival, DOI)

take-home messages: for others to use your code, they need:

- to be able to identify the release (versioning)
- to locate the code (persistent archival, DOI)
- to have no legal barriers (license)

AGH

take-home messages: for others to use your code, they need:

- to be able to identify the release (versioning)
- to locate the code (persistent archival, DOI)
- to have no legal barriers (license)
- to have technical means to install it (package metadata, dependencies, tools)

AGH

take-home messages: for others to use your code, they need:

- to be able to identify the release (versioning)
- to locate the code (persistent archival, DOI)
- to have no legal barriers (license)
- to have technical means to install it (package metadata, dependencies, tools)

- to have documentation! (next week)

AGH

Thank you for your attention!

slides and other course resources at
https://github.com/open-atmos-krk/rse-python-course